Project: Typing Simulator

Report on Individual Project: Typing Simulator

Queen Mary University of London
Name: Akm Abrar Islam
Student ID: 251012618
Module : ECS414U | Object Oriented Programming

Part 1 Report

Commit Title: ***part1; Typist: Implemented Incomplete Functions.***
commit hash: 7e5c331cab0a2b4c87e74beaef250ea0cb2e4fc5

In this commit I fully completed the Typist class as required by part1 specification. All fields (*typistSymbol, typistName, typistAccuracy, progress, isBurntOut, burnoutTurnsRemaining*) are declared private. This means no other class can directly access or modify them using dot notation like Typist1.progress = 100 or Typist1.isBurntOut = false. This design enforces full Encapsulation thereby making sure the object's attributes can only be modified by public getters and setters method of that object.

Class constructor simply accepts typistSymbol, typistName, typistAccuracy and sets them. It initializes other fields to default values.

Overall I wrote these methods:

Getter Methods	**Setter Methods**
getAccuracy()	burnOut()
getProgress()	recoverFromBurnout()
getName	resetToStart()
getSymbol	typeCharacter()
getBurntoutTurnsRemaining()	slideBack()
isBurntOut	setAccuracy()
	setSymbol()

All Getter methods are straightforward -- they simply return the current value of a private field. But Setter methods have valid logic when setting values to private field to protect the integrity of the data that gets stored.

Project: Typing Simulator

Getter Methods

These are simple methods. All of them simply return a private field of the class. Every single getter methods I wrote in Typist class simply returns a field, doesn't include any validation or verification.

Setter Methods and their Validation

All Setter methods takes a value, verifies if it meets certain criteria and then sets them.

- **slideBack(int)** : This method is called when player mistypes. It first checks if 'isBurntOut' field is flagged. Even though in TypingRace class, in advanceTypist method it already makes sure to only call slide back if isBurntOut flag is triggered, I still decided to include it here anyway (defensive programming).

- It also includes (this.progress < 0) check. If a typist is at the very beginning of the passage and mistypes, this validation intercepts the negative value and forcibly clamps progress to 0. This ensures a typist cannot mathematically slide backwards off the starting line.

- **setAccuracy(double)**: Accuracy is defined as a percentage probability between 0.0 and 1.0. This method contains if-else conditions that make sure passed on value is inside that range. If a value greater than 1.0 is passed, it is clamped down to 1.0. If a value less than 0.0 is passed, it is clamped up to 0.0. This prevents runtime mathematical errors in the main TypingRace class, which relies on this value for its Math.random() probability checks.

- **setSymbol(char):** Simple setter method that directly assigns a new char representation to typist. Since it accepts any valid char, no validation required.

- **resetToStart():** This methods acts as a hard reset for object's progress state. It manually resets progress, isBurntOut and burnoutTurnsRemaining fields to their default so same object can be used again.

- **typeCharacter():** Similar to slideback(), this method also includes defensive programming (if (!this.isBurntOut)) to make sure progress is only incremented if the typist is legally allowed to type, thus protecting the state from accidental calls.

- **recoverFromBurntout():** This method contains validation, first it checks if isBurntOut field is true. Only if it's true, it proceeds to decrement burnout counter and check if after decrementing the counter is 0 or not. If it is zero, it flips isBurntOut flag back to false.

Project: Typing Simulator

- **burnOut(int):** This method simply changes isBurntOut field to true and sets burntoutTurnsRemaining to value that's passed on.

How Encapsulation protects data in this class?

1. **Restricts access to fields:** External classes like TypingRace cannot bypass validation. For example they cannot set progress to 999 without calling typeCharacter() or slideBack(). So all the fields of Typist class is protected from unauthorized changes and it's fields can only be changed via public getters and setters methods.
2. **Controlled state changes:** Burnout field can only be entered via burnOut() and exited via recoveryFromBurnout(). There is no setBurnOut(boolean) method, so turn-based recovery rule is enforced automatically.
3. **Ensures consistency:** As all field modifications goes through these public methods, the Typist object can never enter invalid state (eg: negative progress, accuracy outside 0 and 1 etc). This makes whole race simulation more reliable.

Bug Identification & Resolution

Bug 1: The Infinite Race (Progress Overshoot)

Commit Title: part1; TypingRace: Fixed progress overshoot bug
Commit Hash: 8203f2701ffee0ac929de2dbcdb9fcc7f991fecf

The Issue: Ty Posaurus used == for the win condition in raceFinishedBy method in TypingRace class. If a typist advanced by multiple characters at once, they would skip over the exact passage length, causing an infinite loop. So the condition must check if progress is greater than or equal to the passage length.

Solution: Simply change the '==' symbol to '>='.

Bug 2: Missing Visual Mistype Indicator

Commit Title: part1; TypingRace: Print mistyped marker
Commit Hash: 2ddf87b9cd32744425a3cbdae3aa88cc74b57311

The Issue: The UI did not indicate when a typist mistyped and progressbar slid backward, making the race confusing to watch.

The Correct Implementation: The main loop should track progress before and after advanceTypist(). If progress decreases, pass a boolean flag to printSeat() to render a [<] marker.

The Code Fix:

```
// Note down the progress before advancing typist

int[] arr = new int[3];
arr[0] = seat1Typist.getProgress();
arr[1] = seat2Typist.getProgress();
arr[2] = seat3Typist.getProgress();

// Advance each typist by one turn
advanceTypist(seat1Typist);
advanceTypist(seat2Typist);
advanceTypist(seat3Typist);

// Compare the new progress with previous one.
// If progress decreases, flag a boolean variable for each typist.
boolean m1 = seat1Typist.getProgress() < arr[0];
boolean m2 = seat2Typist.getProgress() < arr[1];
boolean m3 = seat3Typist.getProgress() < arr[2];

// Pass on the boolean variable to printRace
// Print the current state of the race
printRace(m1, m2, m3);
// Note printRace method is also adjusted to accept boolean variable.
// It passes on that boolean flag to printSeat which prints the marker.
```

Bug 3: Missing Winner Announcement

Commit Title: part1; TypingRace: Print the winners Name.
Commit hash: bc610aa9dcf91bbd0a638f9fa1a9e601f579e817

The Issue: The startRace() method ended abruptly without declaring who won the race.

The Correct Implementation: When race finishes, we must capture the winning Typist object's name upon completion and print their name.

Bug 4: Unreset Seat 3 when race starts

Commit Title: part1; TypingRace: Reset seat3Typist
Commit hash: 7480f37db5959a80c4e174ce96d9556c0f6c69ff

The Issue: seat3Typist was omitted from the initial resetToStart() calls, meaning they would carry over progress if multiple races were run.

The Correct Implementation: All seated typists must be reset before the while(!finished) loop begins.

Project: Typing Simulator

Bug 5: Fix inconsistent UI prints during simulation

Commit Title: part1; TypingRace: Fix UI print mid-simulation.
Commit hash: 7bf1efe87dfa82798e0b838e4a4a68de5877b263

The Issue: All the print statements during race shows a legend below the progress bar. In there burn out was represented by ‘zz’. But during the race, the actual progressbar represents burntout as “~”.

The Correct Implementation: Simply adjust printRace method’s print statements to correct this inconsistency.

Bug 6: Missing Performance Adjustments

This fix includes two separate commits working together.
Commit 1 Title: part1; Typist: Decrease accuracy when burntout
Commit 2 Title: part1; TypingRace: Implement accuracy handling logic

The Issue: Winning a race or burning out did not affect the typists accuracy over time as required by the specifications.

The Correct Implementation: burnOut() must decrease accuracy (with a floor of 0.1 to prevent the race from turning too long), and startRace() must increase the winner's accuracy by 0.02 at the end.

PS: In commit 2, I also changed the way winner’s name was being printed. Previously I was only retrieving the name of the winner, I changed it to now use the winner object itself. This makes it more flexible, for example in future if I want to make a leaderboard for this game, we just need to pass the winner object to a new function which could directly extract winner’s name, accuracy and other information.

Bug 7: Incorrect mistype probability logic fix.

Commit Title: part1; TypingRace: Invert mistype probability logic
Commit hash: e3d5dea72d1ba68fb843004485cc851bec66a71a

The Issue: Previous logic in advanceTypist() method written by Ty actually made player with more accuracy to mistype more often. Previously probability of a mistype was calculated by multiplying the base chance directly by the typist's accuracy. This caused highly accurate typists (e.g., 0.85) had a 25.5% chance to mistype, while inaccurate typists (e.g., 0.30) had only a 9% chance. The code accidentally rewarded bad typists.

The Correct Implementation: To fix this issue, simply invert the probability math by subtracting it from 1.

Project: Typing Simulator

Other Changes

Commit: part1; TypingRace: Fix accuracy floating-point display output
Commit hash: 13d9f1689aa1fb4e8ac5a9eebf4e1b0db7ccfee6

Issue: Previously when the game was running, It would cause weird long floats that would appear awkward.

Fix: Make sure all UI prints are truncated to 2 decimal place.

Commit: part1; Typist: enforce accuracy validation for constructor.
Commit hash: 3543c421d3d840d31593d66adea61663c73a6c96

Issue: During testing phrase I noticed we could theoritically initialize the Typist class into a invalid state by directly passing on an invalid accuracy value to the Typist class's constructor.

Fix: Make the Typist class's constructor use setAccuracy() method to set accuracy which is already designed to handle validation.

Testing Phase and evidence

All the tests code mentioned below are in a separate file named testTypist.java which is located in: /Part1/testTypist.java.

To compile the test framework,
cd Part1
javac *.java
java testTypist.java

to run all the tests and see the results.

Test A: Progress Floor: Under no circumstances should the progressbar value go negative.

code:

```
Typist t = new Typist('T', "TEST_SUBJECT", 0.5);

System.out.println("Current Progress: " + t.getProgress());
t.slideBack(5); // expected: 0, should not go below zero
System.out.println("Progress after slideBack(5): " + t.getProgress());
```

explanation: This test verifies the defensive programming inside slideBack(). A new typist starts at progress 0 was forced to slide back by value: 5. The output confirms the progress remained exactly at 0, proving the validation

correctly intercepts negative values and prevents the typist from sliding off the track.

proof:

```
========== Test A Started: Progress Floor Test ==========

Current Progress: 0
Progress after slideBack(5): 0

================= Test A Completed =================
```

Test B: Burnout Countdown: When burnout count drops to 0, isBurnOut must get flipped to false.

code:

```
Typist t = new Typist('T', "TEST_SUBJECT", 0.5);

t.burnOut(2);
System.out.println("Triggered Burnout(2). isBurntOut: " + t.isBurntOut() + ", Turns: " + t.getBurnoutTurnsRemaining());

t.recoverFromBurnout();
System.out.println("After 1 recovery call. isBurntOut: " + t.isBurntOut() + ", Turns: " + t.getBurnoutTurnsRemaining());

t.recoverFromBurnout();
System.out.println("After 2 recovery calls. isBurntOut: " + t.isBurntOut() + ", Turns: " + t.getBurnoutTurnsRemaining());
```

Explanation: This test confirms that recoverFromBurnout() handles state transitions properly. After triggering a 2-turn burnout, the output shows the turns decrementing one by one. It also proves that when the counter reaches 0, the isBurntOut boolean automatically flips back to false.

proof:

```
========== Test B Started: Burnout Countdown ==========

Triggered Burnout(2). isBurntOut: true, Turns: 2
After 1 recovery call. isBurntOut: true, Turns: 1
After 2 recovery calls. isBurntOut: false, Turns: 0

================= Test B Completed =================
```

Test C: Accuracy Clamping: accuracy must be between 0 and 1.0.

code:

```
Typist t = new Typist('T', "TEST_SUBJECT", 0.5);

t.setAccuracy(1.5);
System.out.println("Attempted to set accuracy to 1.5. Actual accuracy: " + t.getAccuracy());
```

```
        t.setAccuracy(-0.5);
        System.out.println("Attempted to set accuracy to -0.5. Actual accuracy: "
+ t.getAccuracy());
```

Explanation: These tests verify that the setAccuracy() mutator successfully protects the probability bounds (0.0 to 1.0). Passing 1.5 successfully clamped the value down to 1.0, and passing -0.5 clamped it up to 0.0.

proof:

```
========== Test C Started: Accuracy Clamping ==========

Attempted to set accuracy to 1.5. Actual accuracy: 1.0
Attempted to set accuracy to -0.5. Actual accuracy: 0.0

================= Test C Completed =================
```

Test D: Normal Forward Movement: when typeCharacter() is called, progress must get incremented by 1.

code:

```
        Typist t = new Typist('T', "TEST_SUBJECT", 0.5);

        System.out.println("Initial Progress: " + t.getProgress());
        t.typeCharacter();
        t.typeCharacter();
        System.out.println("Progress after 2 typeCharacter() calls: " +
t.getProgress());
```

Explanation: This test proves the functionality of the typeCharacter() method. Calling it twice successfully incremented the typist's progress by 2, confirming that the typist can move forward.

proof:

```
========== Test D Started: Normal Forward Movement ==========

Initial Progress: 0
Progress after 2 typeCharacter() calls: 2

================= Test D Completed =================
```

Test E: Race Reset Functionality: Test if hard reset works on Typist object.

code:

```
        Typist t = new Typist('T', "TEST_SUBJECT", 0.5);

        // Dirty the state
        t.typeCharacter();
        t.burnOut(3);
```

```
        System.out.println("Before Reset - Progress: " + t.getProgress() + ", isBurntOut: " + t.isBurntOut());

        // Clean the state
        t.resetToStart();
        System.out.println("After Reset  - Progress: " + t.getProgress() + ", isBurntOut: " + t.isBurntOut() + ", Turns: " + t.getBurnoutTurnsRemaining());
```

Explanation: First we create typist object, set some values. Then we call resetToStart(). Then finally print the fields of the object and check if all fields are reset or not.

proof:

```
========== Test E Started: Reset To Start ==========

Before Reset - Progress: 1, isBurntOut: true
After Reset  - Progress: 0, isBurntOut: false, Turns: 0

================= Test E Completed =================
```

Copyright (c) 2026 Akm Abrar Islam
All Rights Reserved.